CODE SPORT

In this month's column, we continue our discussion on testing machine learning applications.

In last month's column, we started with an overview of testing machine learning (ML) programs in general, and we discussed the difficulties in testing ML applications. Since I received a number of responses related to this topic from our readers, I have set up a questionnaire at *https://forms.gle/Yu5oT1ViVBPdVNVC6,* which covers the various aspects of software quality testing of ML applications. I request readers to fill it up. Your responses will help me understand your needs, based on which I will tailor my subsequent columns. You will also get to know about the various aspects of ML testing currently prevalent in research and industry.

Last month, we had primarily looked at what components to test, namely data, the model and implementation framework. Our focus was primarily on testing the 'correctness' property of these components. For example, given the sentence, *"This was a great movie, I absolutely loved it,"* a sentiment analysis model should return a 'positive' class label for this sentence. This would be a correctness test for that model. But there is a caveat here, which I had pointed out briefly in last month's column.

Unlike standard software testing, ML testing has a probabilistic component to it. The classifier returns a set of probabilistic scores associated with each of the output class labels (for a text classification problem). These scores can change based on the changes in training data, code changes in the model, hyper-parameters and changes in the implementation framework. Hence a test sentence which was getting labelled correctly earlier can now get labelled wrongly. Such behaviour in the case of traditional enterprise software would be marked as a regression test failure.

However, in the case of ML systems, this is not a regression test failure, specifically since the outcome is probabilistic. However, if the model's performance on a set of test samples degrades after changes to the model, then it is a regression failure. Therefore, one test input causing a different output from the known correct output is not typically considered as a regression failure in ML testing due to the probabilistic nature of the classifier.

As we pointed out last month, this probabilistic nature of ML systems makes it difficult to design regression test suites. While we can specify an accuracy threshold on the test set and use that as the regression pass threshold, this cumulative measurement across test sets prevents the testing from showing failures on individuals/groups of test inputs. As a simple example, let us assume that our sentiment classification system due to model changes starts classifying all sentences containing the word 'gay' as a negative sentiment. If our regression test suite only contains 5 per cent of such test sentences, the badly behaving model will be able to pass the regression suite accuracy threshold even with this issue.

By now, you may be asking the question, "Wouldn't it be possible to detect this by manual inspection of the failing test cases?" If you see that all the test sentences which have the word 'gay' are marked as negative, it should be obvious. However, test result analysis should be an automated process and not require manual intervention. This makes